UN RIFERIMENTO BM

API di gioco bm, volume 2: suono e musica

Effetti sonori, note, strumenti, brani e pattern riff: ogni funzione audio dell'API delle cartucce, con un primo tutorial.

Indice

Come funziona il suono in bm 4

Cosa fa 4

Il sintetizzatore 4

Il banco dei suoni 5

Da sapere 5

Attenzione 5

Il tuo primo effetto sonoro e la tua prima musica 6

Provalo, passo dopo passo 6

Effetti sonori e musica (il banco della cartuccia) 8

Cosa fa 8

Da sapere 8

Esempio 8

Note dal codice 10

Cosa fa 10

Attenzione 10

Esempi 10

Strumenti, timbro, stanza 12

Cosa fa 12

Le chiavi di `tone()` 13

Esempi 15

Strumenti pronti 15

Il kit della console 16

Profondità in bit 16

Il banco: formato e strumenti 18

Cosa fa 18

Limiti 18

L'editor Sound 18

Sul PC 18

Attenzione 18

riff: musica a pattern 20

Cosa fa 20

Mini-notazione 20

Funzioni 20

Live code 21

Test 22

Punti aperti 23

LEGGIMI

Cos'è. Il volume 2 dell'API di gioco bm: tutto ciò che una cartuccia può fare con suono e musica, da `sfx(n)` ai pattern riff.

Come usarlo. Leggi "Come funziona il suono in bm", poi segui il tutorial "Il tuo primo effetto sonoro e la tua prima musica". Dopo, ogni capitolo copre un'area, con una tabella di tutte le funzioni e brevi esempi.

Cosa manca. Ogni lacuna è segnata con un'etichetta DA COMPLETARE evidenziata; niente è inventato.

Come funziona il suono in bm

L'audio esce in **stereo** a 48 kHz. Sul Pi viaggia su HDMI, quindi lo senti dagli altoparlanti del monitor. Sull'RGB30 esce dalle cuffie o dall'altoparlante. Viene generato in un interrupt, quindi non costa nulla al tuo `_update` .

Cosa fa

Il sintetizzatore mescola 8 voci (numerate da 0 a 7) in virgola mobile, poi applica un compressore e un limitatore morbido all'uscita. Il player gira nello stesso interrupt, ogni 64 campioni (una griglia di 1,3 ms): musica ed effetti sonori restano a tempo qualunque sia il frame rate del gioco.

Ci sono due modi di fare suono, e puoi usarli insieme:

- **il banco dei suoni** della cartuccia: effetti sonori e musica fatti con l'editor Sound della scheda Dev, suonati con `sfx(n)` e `music(n)` ;
- **note** suonate dal codice, una voce alla volta: `note` , `slide` , `arp` ...

Un terzo modo sta sopra le note: **riff** (vedi il capitolo "riff: musica a pattern"), che scrive ritmi e melodie in una riga e le suona a tempo.

Il sintetizzatore

Il sintetizzatore è **hi-fi**: onde senza aliasing, inviluppi esponenziali, un filtro risonante, una stanza (riverbero) e un'eco, e un compressore leggero in uscita. La voce a **8 bit** delle prime versioni resta disponibile: `retro(true)` , la chiave `raw` di `tone()` , oppure *Settings* > *Sound* > *Sound style*.

Forme d'onda (nomi globali, usabili in `note()`):

Onda	Cos'è
`SQUARE`	quadra, con `duty` (la sua larghezza)
`TRIANGLE`	triangolare
`SAW`	dente di sega
`NOISE`	rumore
`SINE`	sinusoide
`METAL`	breve rumore metallico: piatti, campane
`FM`	due sinusoidi, una che piega l'altra: piano elettrico, campane, bassi, ottoni
`PLUCK`	una corda pizzicata: chitarre, arpe

Onda	Cos'è
`SUPERSAW`	tre denti di sega un po' scordati: pad, lead larghi
`ORGAN`	quattro armoniche: organi, flauti
`SAMPLE`	un campione registrato: quello del banco o uno degli otto tamburi della console
`PINK`	un rumore più morbido: vento, mare, fruscio
`BROWN`	anche questo un rumore più morbido: vento, mare, fruscio
`CRACKLE`	click casuali: un vecchio disco, un fuoco
`SID`	SID+ doppio oscillatore: onde combinate, hard sync, modulazione ad anello

Ogni voce ha un inviluppo ADSR, un filtro, una posizione tra sinistra e destra, quanto ne va alla stanza, all'eco e al chorus, e gli effetti del superdough di Strudel: bit crush, una frequenza più bassa, vocali, curve di drive, tremolo, ducking (vedi `tone()`).

Le altezze sono in **Hz** (anche con decimali: `261.63`) o con il **nome della nota**: `"C4"` (do centrale), `"A4"` (440 Hz), `"F#3"`, `"Bb2"`.

Il banco dei suoni

Il banco è la sezione **AUDIO** della cartuccia `.bm`: suoni (strumenti), effetti sonori, pattern e brani, e campioni registrati. Lo fai con l'editor Sound e viene salvato dentro il gioco. Il capitolo "Il banco: formato e strumenti" ha i limiti.

Da sapere

- Otto voci, da 0 a 7. La traccia `t` di un pattern di un brano suona sulla voce `t`.
- I volumi vanno da 0 a 255 per `note()` e da 0 a 1 per `sfx()` e `play()`.
- Il kit di tamburi della console ha otto tamburi fatti dal sintetizzatore all'avvio della console: non c'è nulla di registrato.

Attenzione

- Sul Pi il suono esce solo su HDMI. Un'uscita analogica o I2S non c'è ancora.
- **[DA COMPLETARE: il sintetizzatore hi-fi, lo stile a 8 bit e la profondità in bit non sono ancora verificati sul Pi e sull'RGB30]**

Il tuo primo effetto sonoro e la tua prima musica

Farai un suono con le sole note (nessun banco necessario), poi un piccolo pezzo di musica con riff, poi vedrai come usare il banco.

Provalo, passo dopo passo

1. Apri un progetto di gioco nell'editor di codice, partendo da un `_update` vuoto. **[DA COMPLETARE: come aprire un progetto e farlo girare dalla scheda Dev, passo dopo passo]**
2. Scrivi questo nel codice, poi avvia il gioco:

```
function _update()
  if btnp(4) then
    note(0, 880, 60, SQUARE, 100)   -- un "blip" di 60 ms sulla voce 0
  end
end
```

3. Premi il pulsante **A** (`btnp(4)` : tastiera Spazio, Z o J; croce del DS4; seriale Spazio o `j`). Senti un breve blip. Se non senti nulla, controlla il volume generale con `volume()` : va da 0 a 10.
4. Cambia `880` in `"C5"` . Senti un blip con il nome della nota.
5. Aggiungi un laser: una nota che parte alta e scende. Sostituisci il corpo dell' `if` con:

```
note(1, 1300, 300, SQUARE, 70)   -- parte alta...
slide(1, 300, 250)               -- ...e scende in 250 ms
```

6. Aggiungi un'esplosione: rumore che svanisce. Metti queste due righe dentro l' `if` :

```
envelope(2, 0, 60, 0, 30)        -- un suono che svanisce
note(2, 2500, 300, NOISE, 130)   -- con rumore
```

7. Ora la musica. Metti questo in cima al file e chiama `R.update()` a ogni frame:

```
local R = require "riff"

function _init()
  R.setcpm(30)
  R.play("drums", R.s "kick*4, ~ snare, hat*8")
end

function _update()
  R.update()                     -- a ogni frame: le note dei momenti successivi nella coda
end
```

8. Avvia il gioco. Senti un loop di batteria: quattro kick, un rullante nella seconda metà e otto hat a ciclo.
9. Per fermarlo, chiama `R.hush()` .
10. Per suonare i suoni e i brani della cartuccia, fanne alcuni nell'editor Sound della scheda Dev, che li salva dentro il gioco. **[DA COMPLETARE: come fare un effetto sonoro e un brano nell'editor Sound, passo dopo passo; vedi il manuale dell'editor Sound]**
11. Poi suonali dal codice con `sfx(n)` e `music(n)` :

```
function _init()
  music(0)                          -- brano 0, in loop come deciso nell'editor
end
function _update()
  if btnp(4) then sfx(1) end        -- effetto sonoro 1
end
```

PROVA

Cambia la velocità della batteria: `R.setcpm(30)` diventa `R.setcpm(40)`. Poi aggiungi `R.play("bass", R.note "<c2 a1 f1 g1>" :s "acid")` in `_init` e ascolta i due insieme.

NOTA

Il progetto demo dell'editor Sound `bm/sound/demo.json` ha effetti sonori e tre brani da ascoltare e copiare.

ATTENZIONE

`sfx(n)` restituisce `nil` quando la cartuccia non ha un banco, o quando il numero non è nel banco. Un brano e i suoi effetti richiedono il banco, salvato dentro il gioco.

Effetti sonori e musica (il banco della cartuccia)

Cosa fa

Queste funzioni suonano ciò che hai fatto nell'editor Sound: gli effetti sonori con `sfx` e i brani con `music`.

Funzione	Descrizione
`sfx(n, [v], [semitones], [vol])`	suona l'effetto sonoro `n` del banco; senza `v` sceglie una voce libera, preferendo quelle che la musica lascia vuote. `semitones` lo trasporta (`sfx(0, nil, 12)`: un'ottava su), `vol` 0-1. Restituisce la voce, oppure `nil` (nessun banco, un numero che non c'è)
`sfx(-1, [v])`	ferma l'effetto della voce `v`, oppure tutti
`sfxpos(v)`	l'effetto che suona sulla voce `v` e il suo passo, oppure `nil`
`music(n, [fade_ms], [pos])`	suona il brano `n` dall'inizio (o dalla posizione `pos` della sua sequenza), con dissolvenza in entrata di `fade_ms`
`music(-1, [fade_ms])`	ferma la musica, con dissolvenza
`music()`	mentre suona: brano, posizione, passo, pattern (per stare a tempo); altrimenti `nil`
`tempo(x)`	la musica va `x` volte più veloce (1 = come scritta): accelera quando il gioco si fa difficile
`mute(track, [on])`	spegne (`on`, il predefinito) o riaccende una traccia della musica: strati che vanno e vengono
`volume([level])`	il volume generale 0-10 (con `level` lo cambia). È quello della console: vale per ogni gioco e resta in `bm/config.txt`

Da sapere

- La traccia `t` di un pattern suona sulla voce `t`.
- Finché un effetto sonoro usa una voce, la traccia della musica su quella voce è muta.
- Un banco con un brano che usa le tracce 0-5 lascia libere le voci 6 e 7 per gli effetti.
- `volume()` è il volume della console, non del gioco: vale per ogni gioco.

Esempio

```
function _init()
  music(0)                        -- brano 0, in loop come deciso nell'editor
end
function _update()
  if btnp(4) then sfx(1) end      -- salto
  if got_coin then sfx(0) end
  if boss then tempo(1.2) end     -- più veloce
  if btnp(8) then music(-1, 500) end
end
```

DA COPIARE

```
sfx(0, nil, 12)      -- effetto sonoro 0, un'ottava su
music(1, 500)        -- brano 1, in dissolvenza in entrata di 500 ms
mute(2)              -- spegne la traccia 2 della musica
mute(2, false)       -- e la riaccende
```

Note dal codice

Cosa fa

Queste funzioni suonano una nota su una voce. La forma d'onda e il volume restano quelli dell'ultima nota della voce, quindi basta darli una volta.

Funzione	Descrizione
`note(v, hz, [ms], [wave], [vol])`	suona una nota sulla voce `v` (l'inviluppo riparte); con `ms` finisce da sola, senza resta accesa fino a `noteoff(v)`. `hz` in Hz o con un nome (`"C4"`). `vol` 0-255 (predefinito 128)
`noteoff(v)`	rilascia la nota (parte la fase di rilascio)
`freq(v, hz)`	cambia l'altezza senza ripartire
`slide(v, hz, [ms])`	la nota scivola fino a `hz` in `ms` (predefinito 100): glissandi, sirene, laser
`vibrato(v, [semitones], [hz])`	un vibrato largo `semitones` (per esempio 0.3) a `hz` oscillazioni al secondo (predefinito 6); `vibrato(v)` lo toglie
`arp(v, chord, [ms])`	la nota percorre un accordo, `ms` per nota (predefinito 50): `"major"`, `"minor"`, `"maj7"`, `"min7"`, `"7"`, `"sus2"`, `"sus4"`, `"dim"`, `"aug"`, `"power"`, `"octave"`, oppure una tabella di semitoni (`{0, 4, 7, 12}`); `arp(v)` lo toglie
`hz(note)`	la frequenza di una nota: un numero MIDI (60 = do centrale, 69 = La 440) o un nome (`hz("A4")` = 440)
`envelope(v, a, d, s, r)`	l'inviluppo della voce: attacco, decadimento e rilascio sono tempi 0-255 (0 = istantaneo, 255 = 2 s), il sostegno è un livello 0-255. Predefinito `1, 0, 255, 10`
`duty(v, d)`	la larghezza dell'onda quadra, 0-255 (128 = 50%; 32-64 suona più "nasale")
`playing(v)`	`true` finché la voce suona (rilascio compreso) o un effetto / la musica la tiene
`apu(v, reg, [value])`	legge o scrive un registro grezzo della voce (32 byte per voce, 0-31: `core/src/audio/synth.h`; il registro 10 è 1/256 di Hz, dall'11 il timbro: filtro, posizione, stanza, eco, LFO)

Attenzione

- All'avvio e alla fine della cartuccia le voci si spengono e tornano ai valori predefiniti.
- Più voci forti insieme si sommano: un limitatore le tiene sotto il massimo, ma tieni i volumi intorno a 100-130.

Esempi

```
note(0, 880, 60, SQUARE, 100)          -- un "blip" di 60 ms
note(0, "C5", 60, SQUARE, 100)         -- lo stesso con il nome della nota
envelope(2, 0, 60, 0, 30)              -- un'esplosione che svanisce...
note(2, 2500, 300, NOISE, 130)         -- ...con rumore
note(1, 1300, 300, SQUARE, 70)         -- laser: parte alto...
slide(1, 300, 250)                     -- ...e scende
note(3, "C4", 600, SQUARE, 90)         -- un accordo maggiore arpeggiato
arp(3, "major", 40)
```

Pong, Snake e Star Shooter in `bm/` usano le note (una funzione `jingle` di 10 righe per le melodie; ora c'è `lib.jingle` in bmlib).

PROVA

Suona una nota tenuta e rilasciala: `note(0, "A4")` la fa partire e la tiene accesa, `vibrato(0, 0.3)` la fa oscillare, `noteoff(0)` la rilascia.

Strumenti, timbro, stanza

Cosa fa

`tone()` dà a una voce un suono intero (uno strumento, un filtro, effetti) per i `note()` che seguono. `play()` suona uno strumento come lo suona la musica. `reverb`, `echo` e `chorus` impostano la stanza, l'eco e il chorus a cui tutte le voci mandano il segnale.

Funzione	Descrizione
`tone(v, sound)`	il timbro della voce per i `note()` che seguono: il nome di uno **strumento pronto** (`"epiano"`, `"pluck"`, `"pad"`, `"kick"` ... `instruments()` li elenca) o una tabella in unità semplici (sotto). `tone(v)` torna all'onda quadra di una voce nuova
`play([v], sound, [note], [ms], [vol])`	suona uno strumento come lo suona la musica, con il suo inviluppo d'altezza e il vibrato (il kick che cade, il laser): un nome, una tabella come quella di `tone()` (più `pitch` semitoni da cui partire, `ptime` ms per arrivarci, `vib` cent, `vibhz`, `detune` cent) o un suono del banco (un numero; un nome è prima un suono del banco con quel nome, poi uno strumento pronto, poi un tamburo del kit: `"bd"`, `"sd"`, `"hh"`, `"oh"`, `"cp"`, `"cb"`). `"NAME:N"` è il suono di un campione sull'N-esimo campione dopo il suo (`"kit:2"` il terzo tamburo del kit, l'hi-hat; per un suono del banco i campioni del banco; con giro completo). `note`: un numero MIDI o un nome (`"C4"`, predefinito 60), `ms` 0 = tenuta fino a `noteoff`, `vol` 0-1; senza `v` una voce libera. Restituisce la voce
`instruments([kind])`	gli strumenti pronti: `{ {name=, kind=, about=}, ... }`; `kind`: `"drum"`, `"bass"`, `"keys"`, `"pad"`, `"pluck"`, `"lead"`, `"fx"`
`instrument(name)`	uno strumento con tutti i suoi valori, come bm Sound conserva un suono (`wave`, `a`, `d`, `s`, `r`, `pitch`, `tone` = i 21 byte del timbro...), oppure `nil`; anche un tamburo del kit (`"sd"`) o `"kit:3"`
`reverb([size], [damp], [wet])`	la stanza in cui suonano le voci (0-1: da una stanza piccola a una sala, da brillante a spenta, quanto si sente); restituisce i tre valori. Ogni voce ci manda ciò che dice il suo `reverb` (poco per default: 0.16)
`echo([ms], [feedback], [wet])`	l'eco ping-pong (sinistra, destra): il tempo tra le ripetizioni (680 ms al massimo), quanto ritorna (0-0.95), quanto si sente; restituisce i tre valori. Ciò che entra è l' `echo` di ogni voce

Funzione	Descrizione
`chorus([rate_hz], [depth_ms], [wet])`	il chorus (due ritardi che oscillano, a sinistra e a destra: ampiezza per pad e tastiere): quanto veloce oscilla (0.05-5 Hz, predefinito 0.8), quanto lontano (0-8 ms, predefinito 2.5), quanto si sente (0-1, predefinito 1); restituisce i tre valori. Ciò che entra è il `chorus` di ogni voce
`audio_depth()`	`bits, out` : i bit di ogni campione che la console invia (16, 24 o 32: `sound_depth` in `bm/config.txt` , *Settings* > *Sound* > *Bit depth*, 24 per default) e ciò che l'uscita chiede ora (0: nessuno). Un gioco lo legge, non lo cambia
`audio_samples(["kit"])`	i campioni del banco della cartuccia (la voce `n + 1` è quella che suona `sample = n`) oppure, con `"kit"` , gli otto tamburi del kit: `{ {name=, frames=, rate=, channels=, root=, fine=, loop=, loop_start=, loop_end=, peak=, meter=}, ... }` . `rate` Hz, `root` nota MIDI, `fine` cent, `loop` 0 nessuno, 1 avanti, 2 ping-pong (`loop_start` , `loop_end` in frame), `peak` 0-1 del fondo scala, `meter` 32 picchi 0-1 dall'inizio alla fine
`retro([on])`	ogni voce a **8 bit** come nelle prime versioni (onde ingenue, inviluppi lineari, niente stanza né eco) mentre il gioco gira; restituisce se lo sono (anche Settings può chiederlo)
`audio_time()`	l'orologio del suono in secondi (avanza con i campioni suonati)
`play_at(t, sound, [note], [ms], [vol], [tag])`	una nota al tempo `t` di `audio_time()` : l'interrupt del suono la fa partire, entro 1,3 ms, qualunque sia il frame rate. `sound` come quello di `play()` , `ms` per quanto è tenuta (250 per default), `vol` 0-1, `tag` 1-255 (un gruppo per `play_cancel` , 1 per default). La voce è scelta alla partenza (una libera, una in coda, la nota più vecchia di `play_at` ; mai una di un brano o di un effetto). `true` , oppure `false` quando la coda (160 note) è piena. Lo usa riff
`play_cancel([tag])`	dimentica le note di `play_at` in attesa con quel `tag` (nessuno: tutte) e rilascia quelle che suonano
`play_voices([v, ...])`	le voci che `play_at` può prendere (nessuna: tutte e 8); restituisce quante note aspettano

Le chiavi di `tone()`

Tutte le chiavi sono facoltative; quelle mancanti restano come sono.

Chiave	Valore
`preset`	uno strumento pronto da cui partire

Chiave	Valore
`wave`	`"square"` , `"triangle"` , `"saw"` , `"noise"` , `"sine"` , `"metal"` , `"fm"` , `"pluck"` , `"supersaw"` , `"organ"` , `"sample"` , `"pink"` , `"brown"` , `"crackle"` , `"sid+"`
`vol` , `duty` , `sustain`	0-1
`attack` , `decay` , `release`	ms (fino a 2000)
`cutoff`	il **filtro**: Hz (0 = nessun filtro)
`res` , `filter`	risonanza 0-1; `"lp"` passa-basso, `"bp"` passa-banda, `"hp"` passa-alto, `"notch"`
`keytrack`	`true` : la frequenza di taglio segue la nota (al do centrale è quella data)
`fenv` , `fdecay`	l'inviluppo del filtro: ottave che lo aprono (anche negative), ms per richiudersi
`lfo` , `wah` , `pwm`	un lento ondeggiare (Hz) che muove il filtro (`wah` ottave) e la larghezza della quadra (`pwm` 0-1)
`drive` , `noise`	saturazione morbida prima del filtro, rumore bianco aggiunto (0-1)
`pan`	-1 sinistra, 0 centro, 1 destra
`reverb` , `echo`	quanto la voce manda alla stanza e all'eco (0-1)
`ratio` , `depth` , `mdecay` , `feedback`	FM: il rapporto del modulatore (1, 2, 3.5...), la profondità (radianti, 0-8), in quanti ms svanisce, feedback 0-1
`bright` , `ring`	PLUCK: quanto è brillante la corda e quanto risuona (0-1)
`spread`	SUPERSAW: quanto sono scordate le tre seghe (0-1)
`bars`	ORGAN: i quattro tiranti `{8, 6, 3, 2}` (0-15)
`sample` , `begin` , `reverse`	SAMPLE: quale campione (un numero del banco, un nome del banco, un tamburo del kit `"bd"` `"sd"` `"hh"` `"oh"` `"cp"` `"rim"` `"tom"` `"cb"` , oppure `"kit:N"`), dove parte (0-1 della sua lunghezza), `true` all'indietro. La nota rispetto alla nota base del campione ne fissa la velocità
`density`	CRACKLE: quanti click (0-1)
`color`	il colore del `noise` mescolato: `"white"` , `"pink"` , `"brown"` , `"crackle"`
`curve`	la curva del `drive` : `"soft"` (come prima), `"hard"` , `"fold"` , `"sine"` , `"asym"` (armoniche pari, una valvola), `"cubic"`
`vowel`	i formanti di una vocale dopo il filtro: `"a"` , `"e"` , `"i"` , `"o"` , `"u"` , `""` nessuna

Chiave	Valore
`crush` , `coarse`	bit crush: i bit tenuti (1-15; 0 nessuno); la frequenza divisa (1-16: ogni valore tenuto per quel numero di campioni)
`trem`	tremolo: l' `lfo` sul volume, quanto profondo (0-1)
`duck`	ducking: questa voce spinge giù le altre finché suona (0-1): un kick che fa pompare un pad, un effetto che abbassa la musica
`chorus`	quanto la voce manda al chorus (0-1; lo imposta `chorus()`)
`analog`	`true` : saturazione analogica non lineare dentro il filtro risonante
`bit16`	`true` : quantizzazione digitale della voce a 16 bit (24 bit hi-fi è il default; `raw` è 8 bit)
`s6581`	`true` : carattere 6581 con onde combinate bitwise e DAC a 12 bit per `"sid+"`
`raw`	`true` : questa voce è a 8 bit (gli effetti sopra sono ignorati, come in `retro`)

Ogni chiave nuova a 0 (o mancante) è il suono di prima. `play()` e `play_at()` accettano le stesse chiavi nella loro tabella.

Esempi

```
tone(0, "epiano")                                  -- un piano elettrico per note()
note(0, "E4", 400)
tone(1, { preset = "bass", cutoff = 300, res = 0.7 })   -- un basso più scuro, che risuona
play(nil, "kick", "C2")                            -- il kick, su una voce libera
play(nil, { wave = "noise", cutoff = 900, fenv = -2, fdecay = 300, decay = 400, sustain = 0 }, "C3")
play(nil, "sd")                                    -- il rullante del kit (un campione)
play(nil, { preset = "kit", sample = "hh", reverse = true })  -- l'hi-hat all'indietro
tone(2, { preset = "lead", crush = 4, coarse = 3 }) -- un lead rotto, 4 bit
tone(3, { preset = "pad", vowel = "o", chorus = 0.6 })  -- un coro che canta "oh"
play(nil, { preset = "kick", duck = 0.8 }, "C2")    -- il kick abbassa tutto il resto
reverb(0.8, 0.5, 1)                                -- una cattedrale (Yharnam)
echo(375, 0.4, 1)                                  -- un'eco a tempo a 80 BPM
chorus(0.5, 4, 1)                                  -- un chorus più lento e più largo
retro(true)                                        -- il suono a 8 bit di prima
```

Strumenti pronti

Sono 55 (`core/src/audio/presets.c`):

Tipo	Strumenti
tamburi	`kick` , `punch` , `snare` , `clap` , `hat` , `openhat` , `tom` , `rim` , `crash` , `cowbell` , `shaker` ; a 8 bit `chipkick` , `chipsnr` , `chiphat` ; `kit` (i tamburi del kit come campioni: `"kit:0"` ... `"kit:7"`); `pump` (il kick del kit che abbassa il resto: una pompata sidechain)
bassi	`bass` , `acid` , `sub` , `fmbass` , `pickbass` , `bitbass` (schiacciato a 4 bit)
tastiere	`epiano` , `organ` , `bell` , `marimba` , `glock` , `rhodes` (con tremolo e chorus)
pad	`pad` , `strings` , `warm` , `glass` , `lush` (supersaw nel chorus), `choir` (canta "aah")
corde	`pluck` , `guitar` , `harp`
lead	`lead` , `sawlead` , `flute` , `brass` , `triangle` , `chip` , `chiptri` , `solo` (drive caldo, vibrato, eco)
effetti	`laser` , `blip` , `boom` , `wind` , `vinyl` (il crepitio di un disco)
SID+	`sidlead` , `sidsync` , `sidbass` , `sidring` , `sidpad`

Gli stessi strumenti sono nel menu *Instrument...* di bm Sound.

Il kit della console

Otto tamburi fatti dal sintetizzatore all'avvio della console (nulla di registrato), suonati come campioni: `bd` kick, `sd` rullante, `hh` hi-hat chiuso e `oh` aperto, `cp` battito di mani, `rim` , `tom` , `cb` cowbell (`"kit:0"` ... `"kit:7"` in quest'ordine).

- `play(nil, "cp")` e `tone(v, { wave = "sample", sample = "oh" })` li suonano.
- `rim` e `tom` per nome sono gli strumenti del sintetizzatore; quelli del kit sono `"kit:5"` e `"kit:6"` .
- Nella modalità a 8 bit suonano ancora come tamburi (8 bit, il frame più vicino).

Profondità in bit

Il mix è in float e viene arrotondato una sola volta, con dither, alla profondità che l'uscita porta: `sound_depth=16|24|32` in `bm/config.txt` (*Settings* > *Sound* > *Bit depth*, 24 per default). L'HDMI del Pi e il codec dell'RGB30 convertono al massimo 24 bit; `16` è esattamente il vecchio percorso. `audio_depth()` dice a un gioco quale è.

ATTENZIONE

[DA COMPLETARE: l'uscita a 24 e 32 bit non è ancora stata provata sulla console RGB30; se a 24 suona male, `sound_depth=16` è esattamente il vecchio percorso]

PROVA

Chiama `tone(0, "epiano")` e `note(0, "E4", 400)`. Poi `reverb(0.8, 0.5, 1)` e suona di nuovo la nota: senti la stessa nota in una grande sala.

Il banco: formato e strumenti

Cosa fa

Il banco è la sezione **AUDIO** del `.bm` (formato in `core/src/audio/player.h`, versione 2, o 3 con i campioni; la versione 1 si legge ancora).

Limiti

- fino a 32 suoni (strumenti, ognuno con il suo timbro: filtro, posizione, stanza, eco, LFO, effetti);
- 64 effetti sonori;
- 64 pattern;
- 8 brani (ognuno con l'eco a tempo, in passi, e la dimensione della stanza);
- fino a 64 **campioni**: 8, 16, 24, 32 bit o float, mono o stereo, qualunque frequenza, con una nota base e un loop (avanti o ping-pong); 16 MB a banco, circa 174 s di mono a 48 kHz.

L'editor Sound

Il banco si fa con l'**editor Sound** (scheda Dev), che apre un gioco e salva i suoi suoni dentro di esso. Il *Samples...* del suo menu elenca i campioni del banco e quelli del kit con lunghezza e picco, e ne mette uno nel suono (l'onda SAMPLE, con SAMPLE e START tra le sue impostazioni). Il piè di pagina del menu mostra la profondità dell'uscita (`output 24-bit`).

Sul PC

Comando	Cosa fa
`core/scripts/bmaudio.py unpack game.bm -o sounds.json`	estrae il banco come JSON leggibile (i campioni come file WAV accanto)
`mkbm.py --audio sounds.json`	rimette il JSON in una cartuccia
`make wav BANK=sounds.json SONG=0`	suona il banco in un WAV (`BITS=24` , `32` o `f32` per un file più profondo; 16 per default)
`bmhost --wav-bits 24`	registra il suono di un gioco a 24 bit

Un file WAV entra in un banco dal JSON, e un suono lo suona per nome:

DA COPIARE

```
"samples": [{"name": "voice", "wav": "voice.wav", "root": "C4", "loop": "fwd"}]
"wave": "sample", "sample": "voice"
```

Attenzione

- Un pacchetto di suoni in `/bm/sounds` (un `.bm` con la sola sezione AUDIO) si può ancora leggere; bm Sound salva i nuovi come file `.bms` in `/bm/lib` .
- **Importa da...** di bm Sound copia insieme al suono il campione di un suono SAMPLE e rinumera il `MOD1` del suono. Un campione con gli stessi byte non viene copiato due volte. I tamburi del kit non hanno bisogno di copia. Un banco contiene al massimo 64 campioni.

- **Import WAV or MIDI...** di bm Sound (Ctrl+W) legge i file `.wav` e `.mid` da `/docs` sulla scheda SD con `doc_list` e `doc_read` : lo strumento non ha bisogno di permesso. Un WAV diventa un campione del banco (interi da 8 a 32 bit e numeri in virgola mobile a 32 o 64 bit, mono o stereo, fino a 4 MB, sopra 48000 Hz portato a 48000 Hz, nota base e loop di un chunk `smpl` mantenuti). Un file MIDI diventa una canzone di pattern da 64 passi, con il canale delle percussioni sui tamburi del kit.

riff: musica a pattern

Cosa fa

`local R = require "riff"` : ritmi e melodie scritti in una riga, come in TidalCycles e Strudel, suonati a tempo dall'interrupt del suono (`play_at`) con gli strumenti pronti e i suoni del banco per nome. Il tempo si conta in **cicli** (una battuta di quattro tempi; 2 s all'inizio). Tutto il linguaggio dei pattern è nel manuale di riff.

```
local R = require "riff"

function _init()
  R.setcpm(30)
  R.play("drums", R.s "kick*4, ~ snare, hat*8")
  R.play("bass", R.note "<c2 a1 f1 g1>" :s "acid" :lpf(R.sine:range(300, 1800):slow(4)))
end

function _update()
  R.update()                      -- a ogni frame: le note dei momenti successivi nella coda
end
```

Mini-notazione

Mini-notazione	Cosa fa
`a b c d`	una sequenza nel ciclo
`~` `-`	una pausa
`[a b]`	un gruppo in una parte
`a, b`	insieme
`<a b>`	uno a ciclo
`a*2` , `a/2`	più veloce, più lento (anche `a*<2 4>`)
`a!3` , `a@3` , `a _ _`	ripetuto, più lungo
`a?` , `a?0.3`	a caso
`a(3,8,2)`	un ritmo euclideo
`{a b c}%4`	poliritmo
`a | b`	uno dei due, a ogni ciclo
`kick:2`	una variante (due semitoni su)
`kit:2` , `bd sd hh`	il tamburo 2 del kit (campioni, alla loro velocità); i tamburi del kit per nome, come in Strudel

Funzioni

Funzione	Descrizione
`R.s(p)` , `R.note(p)` , `R.n(p)` , `R.chord(p)`	strumenti, note (`c4` = 60, ottava 3 se manca), gradi di una scala, accordi (`"<Am F C G7>"`)
`R.seq` , `R.cat` , `R.stack` , `R.timecat` , `R.arrange` , `R.run(n)`	in sequenza, un ciclo ciascuno, insieme, con pesi, sezioni di più cicli, `0..n-1`
`R.sine` , `R.cosine` , `R.saw` , `R.tri` , `R.square` , `R.rand` , `R.perlin` , `R.irand(n)` , `R.choose(...)`	segnali 0..1
`:fast` , `:slow` , `:early` , `:late` , `:rev` , `:ply` , `:iter` , `:palindrome`	tempo
`:every(n, f)` , `:lastOf` , `:sometimes(f)` , `:often` , `:rarely` , `:degradeBy(x)` , `:someCycles`	cambiamenti ogni tanto, a caso
`:euclid(k, n, r)` , `:struct(p)` , `:mask(p)` , `:segment(n)` , `:chunk(n, f)` , `:linger(x)` , `:swing(n)`	struttura
`:off(t, f)` , `:superimpose(f)` , `:layer(...)` , `:jux(f)`	strati, sinistra e destra
`:add` , `:sub` , `:mul` , `+` , `:transpose` , `:scale("C:minor")` , `:arp("updown")` , `:range(a, b)`	note e numeri
`:s` , `:gain` , `:legato` , `:lpf` , `:hpf` , `:bpf` , `:res` , `:pan` , `:room` , `:delay` , `:attack` , `:decay` , `:sustain` , `:release` , `:shape` , `:vib` , `:fm` , `:raw` , `:crush` , `:coarse` , `:vowel` , `:chorus` , `:trem` , `:duck` , `:begin` , `:tone{...}`	il suono di ogni nota (le unità di `tone()` ; `:tone{curve = "fold", color = "pink"}` per il resto)
`R.play(name, p)` , `R.stop(name)` , `R.hush()` , `R.update()`	suona (con un nome), ferma, da chiamare in `_update`
`R.setcps(x)` , `R.setcpm(x)` , `R.bpm(x)`	la velocità
`R.code(text)`	live code: le funzioni di riff sono globali, ogni globale a cui dai un pattern suona con il suo nome, quelle di prima non nominate di nuovo si fermano; `true` oppure `nil` e l'errore
`R.bake(p, {cycles=, steps=16})` , `R.piece(t)`	il pattern come brano per il banco (la forma di `ai.music`), un brano di `ai.music` come pattern
`R.playing()` , `R.get([name])` , `R.errors()` , `R.active()` , `R.voices(v, ...)`	i nomi che suonano, i loro pattern, gli errori, dove sono scritte le note che suonano, le voci che può usare

Live code

```
R.code [[
setcpm(28)
local verse = chord "<Am F C G>"
pad   = verse :s "pad" :room(.6) :gain(.7)
arp   = verse :s "pluck" :arp("updown") :fast(4)
drums = s "kick ~ ~ kick, ~ snare, hat*8?" :every(4, fast(2))
]]
```

```
R.code [[
beat  = s "bd ~ [~ bd] ~, ~ cp, hh*8" :crush(8)        -- i campioni del kit, 8 bit
vox   = note "<a3 f3>" :s "choir" :vowel("o") :chorus(.5)
]]
```

In **bm Code**, Ctrl+Enter suona il codice (la scheda, o il blocco `riff.code [[ ]]` sotto il cursore) e illumina le parole delle note che suonano; Ctrl+. lo ferma. In **bm Sound**, F7 (*Riff...*) suona una riga e Ctrl+Enter la mette nel banco come brano.

Test

`make test-riff`, e i test QEMU `test_riff_code` e `test_sound_riff`.

Punti aperti

- **Come funziona il suono in bm**: il sintetizzatore hi-fi, lo stile a 8 bit e la profondità in bit non sono ancora verificati sul Pi e sull'RGB30
- **Il tuo primo effetto sonoro e la tua prima musica**: come aprire un progetto e farlo girare dalla scheda Dev, passo dopo passo
- **Il tuo primo effetto sonoro e la tua prima musica**: come fare un effetto sonoro e un brano nell'editor Sound, passo dopo passo; vedi il manuale dell'editor Sound
- **Strumenti, timbro, stanza**: l'uscita a 24 e 32 bit non è ancora stata provata sulla console RGB30; se a 24 suona male, `sound_depth=16` è esattamente il vecchio percorso